```
$touch ~/Templates/New\ Document
```

Here…

i) The back slash (‘\’) indicates that the following name contains space.

ii) ‘touch’ creates an empty file.

After this, ‘New Document’ will be one of your right click options.

—Gunasekar Duraisamy, dg.gunasekar@gmail.com

Killing a number of processes with one command

Let’s suppose you have too many processes running and you want to kill all of them at one go, e.g., lots of Vim are running and you want to close all of them together. Here is a small tip that will help you do this.

```
$ps -ef | grep ‘vim’ | awk ‘{print $2}’ | xargs kill -9
```

You can modify the above command to kill all the Vim instances or any one that you choose to, like ‘vim mohit’.

To do the latter, use the following command:

```
$ps -ef | grep ‘vim’| grep -v “mohit” | awk ‘{print $2}’
| xargs kill -9
```

—Mohit, mohitraj.cs@gmail.com

Emptying a file without deleting and renaming it again

This tip is for those who want to empty a file without deleting and renaming it again, which is a tedious process.

To clean the contents of the file, you can use the following command:

```
$cat filename > filename
```

Let’s suppose you created a file called *log.txt*, and want to clean it completely without deleting it. Use the command given below:

```
$cat log.txt > log.txt
```

In the above scenario, the file of the same name is created first, so the contents of the file are gone. When we do what’s shown above, you don’t have to close the file using CTRL+C; instead, it ends automatically and an empty file with the same name is created.

Another way of clearing the contents of a file is by using the ‘echo’ command. This is used for printing an output on the shell. But it has other functionalities too. If you look into the man page of Echo, you will see that *-n* actually does not output the trailing new line.

So, when we use the following command, it takes the input of the ‘echo’ which is empty and the *-n* does not move to the new line; hence, the cursor of the file starts from the beginning instead of the next line. We are taking that input and placing it in the file name. Hence, the file gets empty.

The general way is:

```
$echo -n > filename
```

In our example, the command to be given is:

```
$echo -n > log.txt
```

Running command(s) at specified time intervals

Systems engineers often want to execute command(s) at specified time intervals for monitoring purposes. Of course, we can use a combination of the infinite *while* loop and *sleep* command to achieve this goal, but why write three lines of script when we can use the simple *watch* command.

The syntax is:

```
watch -n <seconds> <command>
```

If you are running some batch and want to monitor the free space on the */tmp* mount, use the following command:

```
$watch -n 60 “df -h /tmp”
```

Note: You might need to put your command in quotes if it has ‘-’ or any other special/escape characters. Also, you can use multiple commands separated by a ‘;’ (semi colon).

—Kaushik Patil, kaushik.patil.10791@gmail.com

END

Share Your Open Source Recipes!

The joy of using open source software is in finding ways to get around problems—take them head on, defeat them! We invite you to share your tips and tricks with us for publication in *OSFY* so that they can reach a wider audience. Your tips could be related to administration, programming, troubleshooting or general tweaking. Submit them at *www.opensourceforu.com*. The sender of each published tip will get a T-shirt.